

PHP 8.6 and Other stuff

PHP 8.6 and Other stuff



PHP 8.6 and Other stuff



PHP 8.6 and Other stuff



PHP 8.6 and Other stuff

CALL FOR SPEAKERS

<https://phptek.io>



PHP TEK 2027

19TH ANNUAL

APRIL 27TH - 29TH, 2027

SHERATON SUITES, CHICAGO



PHP Tek 2027

event starts
27 Apr 2027

event ends
29 Apr 2027

location
Sheraton O'Hare
Chicago, Illinois, United States

website
phptek.io

Welcome to PHP Tek where we combine leadership, expertise, and networking in one event. A relaxing atmosphere for tech leaders and developers to share, learn and grow professionally while also providing you with the knowledge to solve your everyday problems. Join us on the journey to be excited about work again!

We are excited to be returning to Chicago and the Sheraton O'Hare Hotel. This hotel is conveniently located near the airport and offers a complimentary airport shuttle.

Call for Speakers

Call opens at 12:00 AM
01 Jun 2026

Call closes at 11:59 PM
31 Oct 2026

Call closes in **Central Daylight Time (UTC-05:00)** timezone.
Closing time in your timezone (America/New_York) is **1 Nov 2026 12:59 am**.

Please give us a brief overview of your presentation idea, what level of developer you are targeting, any prior knowledge needed, etc...

IMPORTANT:

- Please submit more than one presentation idea. We often ask speakers to give two presentations.
- Please DO NOT submit presentations you've already give at PHP Tek in the past.
- Review the Speaker Package at the bottom of this detail section.

If your submission is not accepted, we will be extending our Blind Early Bird pricing to you after the approval process.

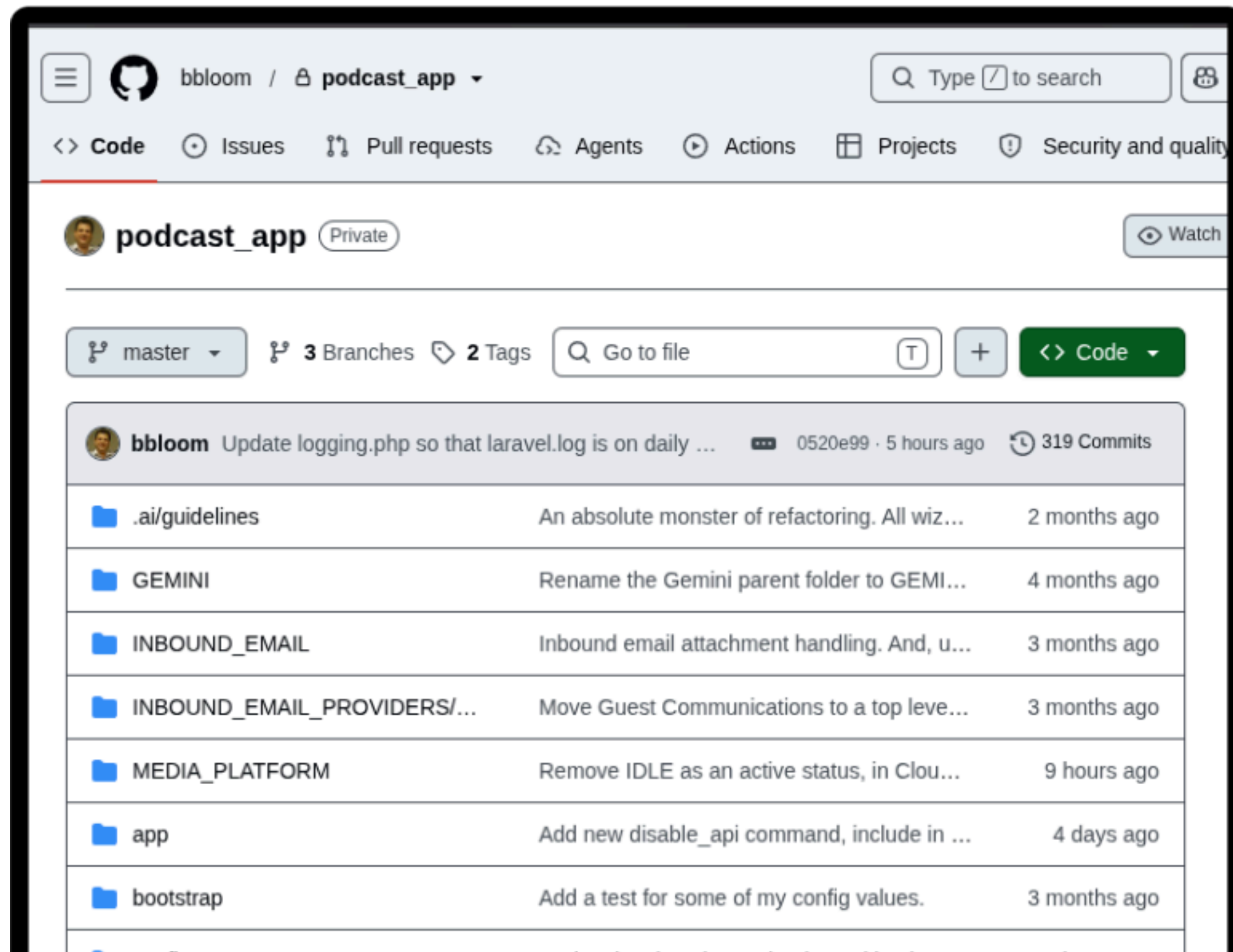
Our Commitment

PHP Tek is dedicated to providing a harassment-free conference experience for everyone, regardless of gender, gender identity and expression, age, sexual orientation, disability, physical appearance, body size, race, ethnicity, religion (or lack thereof), or technology choices.

Expected Behavior

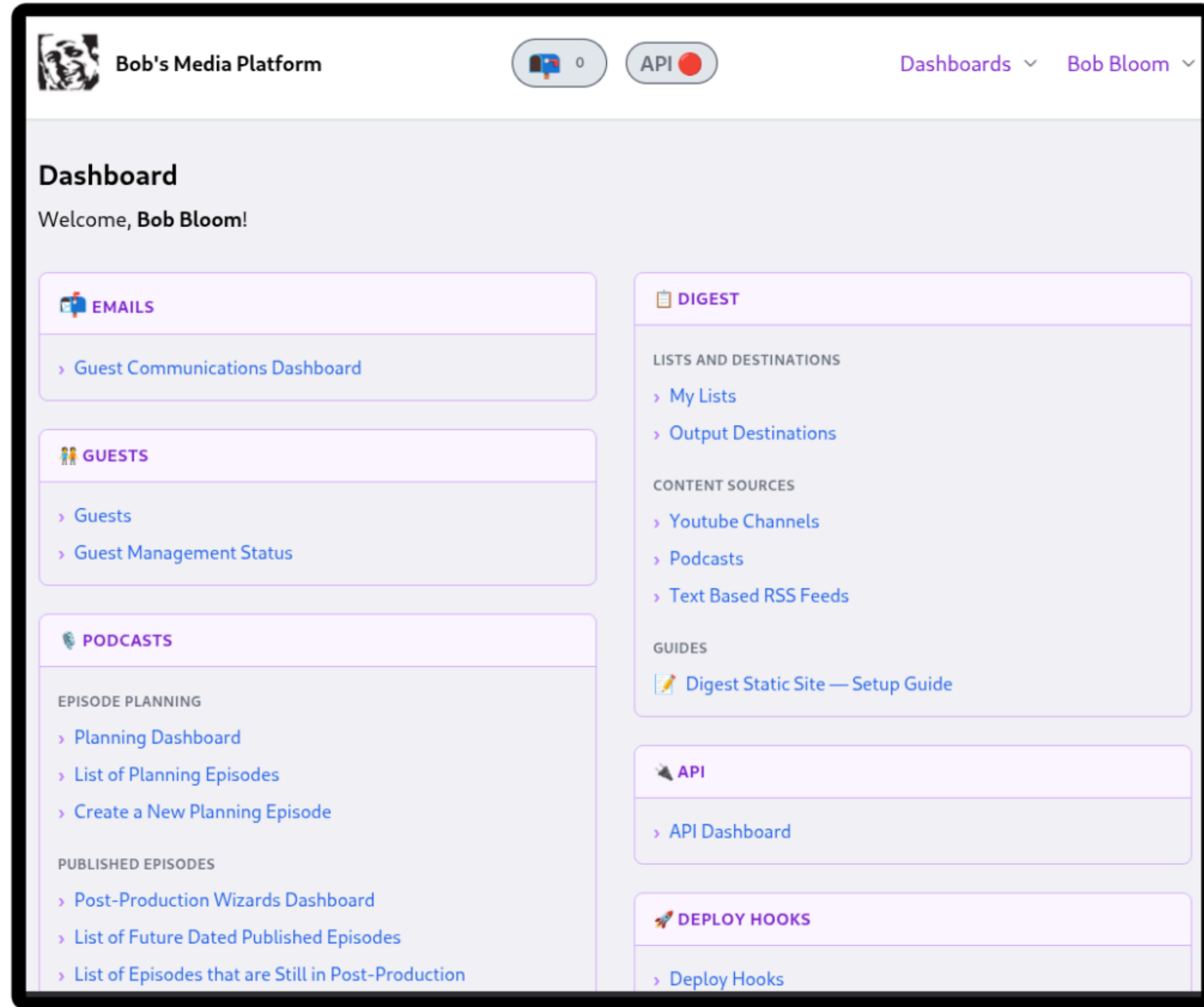
For the first time ever,
I did a “call for speakers”.
For PHP Tek 2027

PHP 8.6 and Other stuff



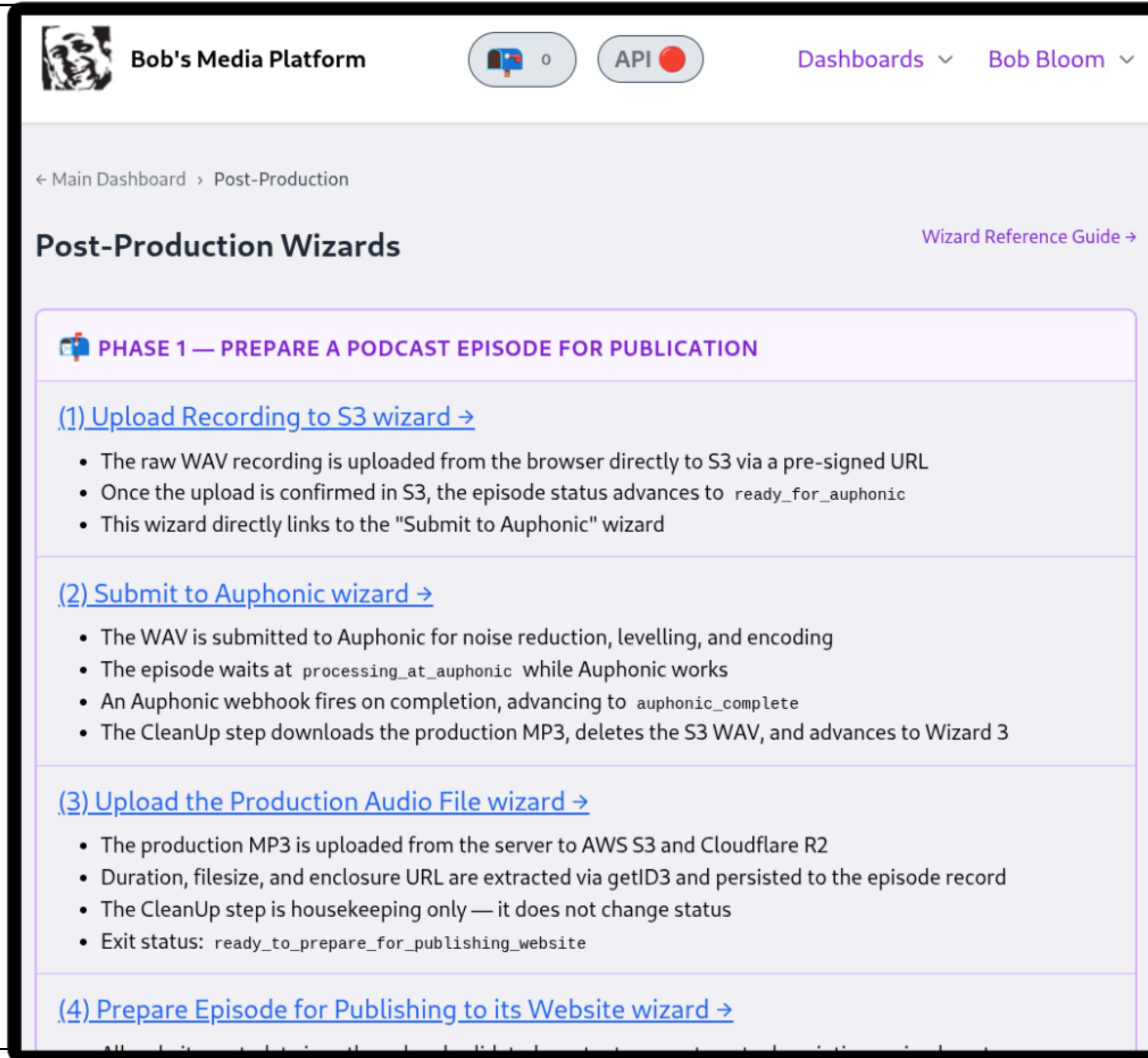
I worked on my
custom
podcasting
app

PHP 8.6 and Other stuff



Main Dashboard

PHP 8.6 and Other stuff



The screenshot shows the 'Post-Production Wizards' section of the 'Bob's Media Platform'. The header includes a user profile, navigation links, and a status bar. The main content area is titled 'Post-Production Wizards' and contains a list of four wizards for preparing a podcast episode for publication. Each wizard is described with a list of steps and actions.

Bob's Media Platform

← Main Dashboard > Post-Production

Post-Production Wizards

Wizard Reference Guide →

PHASE 1 — PREPARE A PODCAST EPISODE FOR PUBLICATION

(1) [Upload Recording to S3 wizard →](#)

- The raw WAV recording is uploaded from the browser directly to S3 via a pre-signed URL
- Once the upload is confirmed in S3, the episode status advances to `ready_for_auphonic`
- This wizard directly links to the "Submit to Auphonic" wizard

(2) [Submit to Auphonic wizard →](#)

- The WAV is submitted to Auphonic for noise reduction, levelling, and encoding
- The episode waits at `processing_at_auphonic` while Auphonic works
- An Auphonic webhook fires on completion, advancing to `auphonic_complete`
- The CleanUp step downloads the production MP3, deletes the S3 WAV, and advances to Wizard 3

(3) [Upload the Production Audio File wizard →](#)

- The production MP3 is uploaded from the server to AWS S3 and Cloudflare R2
- Duration, filesize, and enclosure URL are extracted via getID3 and persisted to the episode record
- The CleanUp step is housekeeping only — it does not change status
- Exit status: `ready_to_prepare_for_publishing_website`

(4) [Prepare Episode for Publishing to its Website wizard →](#)

Post Production Dashboard

PHP 8.6 and Other stuff

Guest Communications

NEW EMAILS

› New Emails

REPLY LATER

› Reply Later

SEND

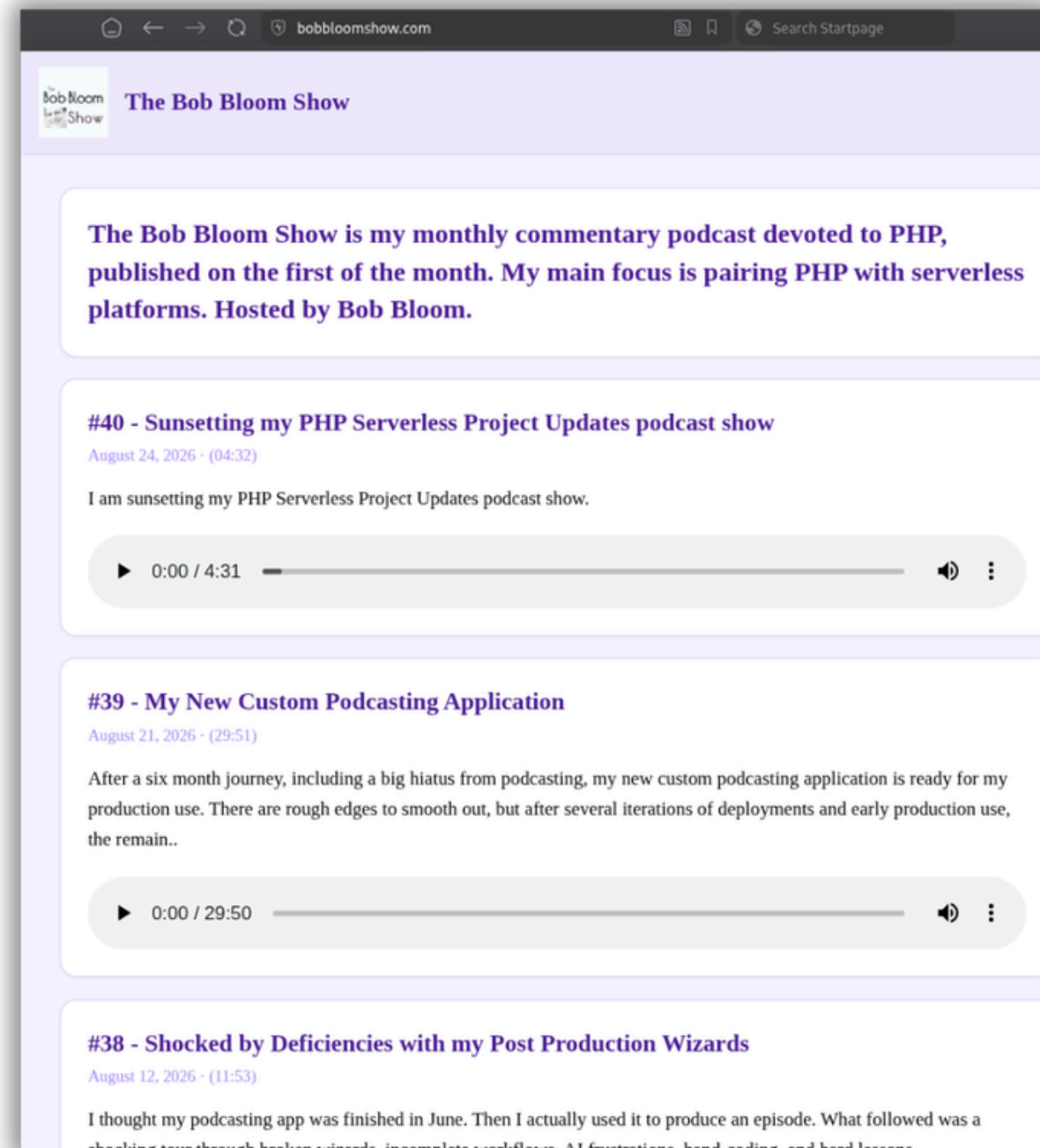
› Send a New Email

MILESTONE

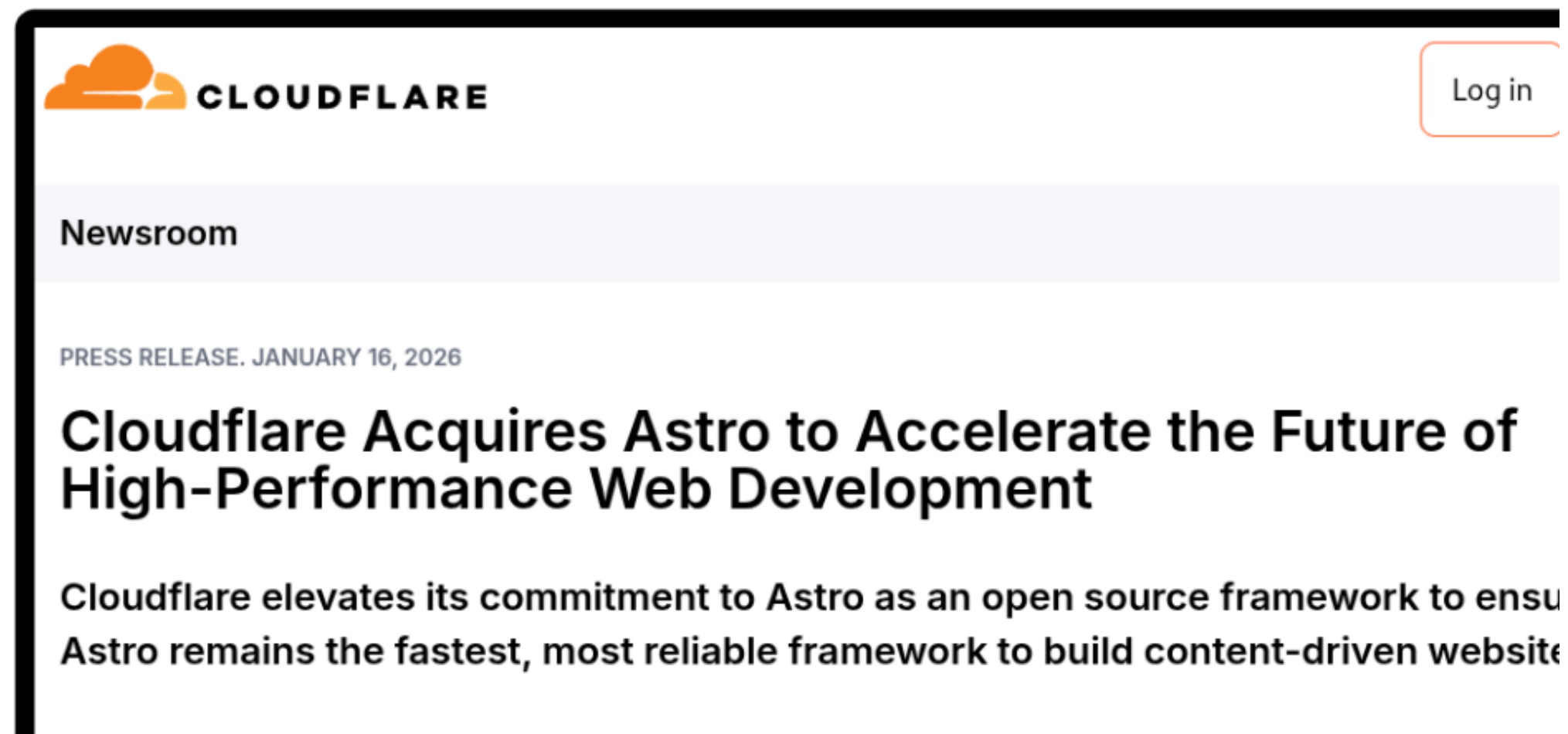
- › Milestone Emails
- › Initial Contact Wizard
- › Invitation Wizard
- › Set Up Interview Wizard
- › Reschedule Interview Wizard

Inbound/Outbound
Email
+
Guest Managment

PHP 8.6 and Other stuff



Completely re-did
my sites
with Cloudflare Pages
with Astro



PHP 8.6 and Other stuff



Arranged
my first interview.

Look for it
later this month.

Miscellaneous Stuff



PHP 8.6 and Other stuff

We rarely, if ever, go through Laravel Framework releases

v13.30.0

 **github-actions** released this 2 weeks ago · 82 commits to 13.x since this release  Immutable  v13.30.0  9cd

- [12.x] Backport [#60908](#): Avoid quadratic wildcard rule expansion by [@matteo-palazzo](#) in [#61232](#)
- Fix resource loading by [@taylorotwell](#) in [#61323](#)
- [13.x] Output the worker stop reason in queue:work by [@jackbayliss](#) in [#61339](#)
- Fix incorrect \$startTime type in Worker::stopIfNecessary docblock by [@il-m-tomita](#) in [#61347](#)
- [13.x] Ability to opt out of vendor/default commands in DevCommands by [@jackbayliss](#) in [#61344](#)
- [13.x] Confine `Storage::path()` to the configured disk root by [@KIKOmanasijev](#) in [#61343](#)
- [13.x] Support Microsoft SQL Server DSN connection strings by [@HenkPoley](#) in [#61341](#)
- [13.x] Restore the facade application after route:cache boots a fresh application by [@reianseni](#) in [#61346](#)
- [13.x] Gracefully handle missing param value or strings with request clamps by [@kylemilloy](#) in [#61355](#)
- [13.x] Restore default database connection when seeding fails by [@sayful1411](#) in [#61354](#)
- [13.x] Cloud enhancements by [@timacdonald](#) in [#61360](#)
- [13.x] Allow retrieving command by name without resolving all commands by [@timacdonald](#) in [#61361](#)
- [13.x] Expose methods by [@timacdonald](#) in [#61362](#)
- [13.x] Add basic test for query explain by [@imanghafoori1](#) in [#61363](#)
- [13.x] Add totalXSize methods to Cloud Queue by [@jackbayliss](#) in [#61352](#)
- [13.x] Add `chunkBy` to collections by [@JosephSilber](#) in [#61357](#)
- [13.x] spl_object_id over hash by [@jackbayliss](#) in [#61372](#)
- [13.x] Add createPayloadUsing directly to the queue manager by [@jackbayliss](#) in [#61367](#)
- [13.x] Set visibility on files that only exist on a read-through disk's fallback by [@sulimanbenhalim](#) in [#61375](#)
- [12.x] Disable html by default on Laravel exception page tooltip by [@crynobone](#) in [#61381](#)

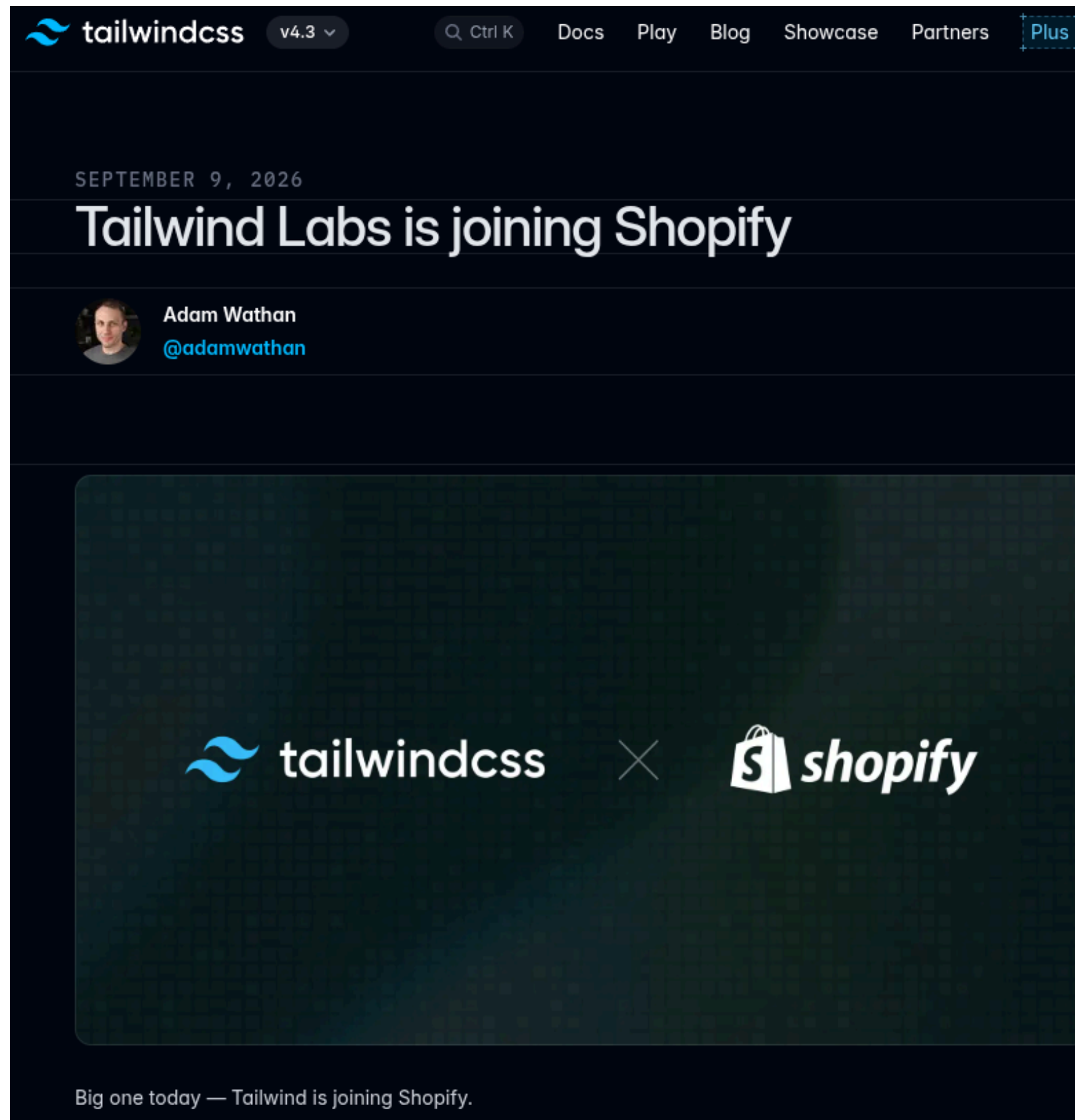
PHP 8.6 and Other stuff

**I recommend
subscribing to the
“Laravel”
YouTube channel.**

**And, then, doing
Lightening Talks!**



PHP 8.6 and Other stuff

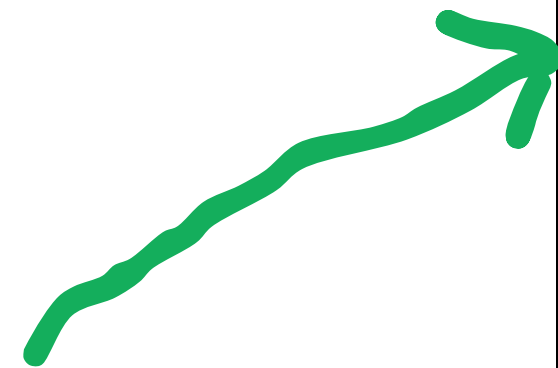


- stable long-term home
- actively maintained
- nothing changes with Tailwind CSS or any of our other open-source projects
- on the commercial side, we'll no longer be trying to grow the business around Tailwind

PHP 8.6 and Other stuff

TypePHP by Swoole

PHP is written in C

The screenshot shows the Swoole website's 'Compile PHP to Native Binaries' page. The navigation bar at the top includes links for Home, Download, Docs, Blog, Features, Support, and Back to main site, along with a language switcher (中文) and a GitHub link. The main heading is 'Compile PHP to Native Binaries'. Below it, a paragraph explains that the TypePHP compiler translates PHP source into C++ and then compiles it into native machine code, eliminating interpretation overhead. There are two buttons: 'Quick Start' and 'Read the Docs'. At the bottom, there are three columns: 'x150' (Faster computation), 'Native' (Native machine code), and 'Cross-platform' (Linux / macOS / Windows / WASM). A terminal window on the right shows the following commands:

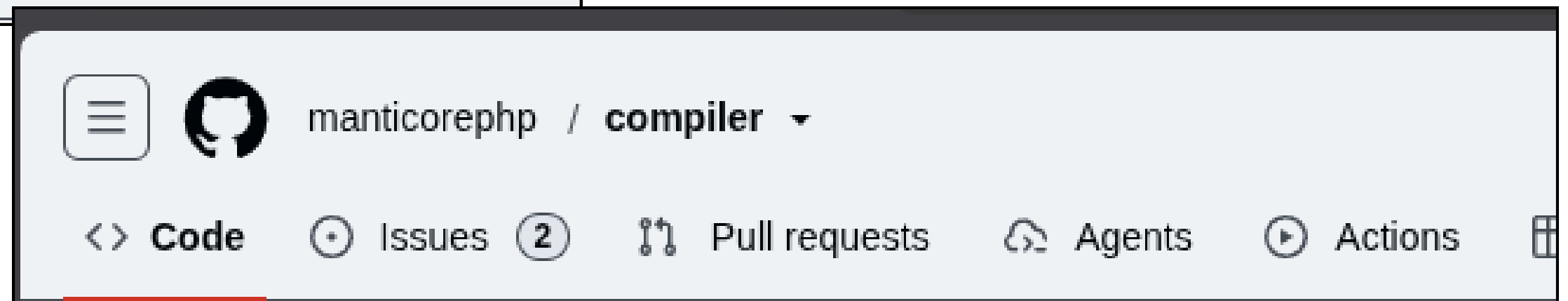
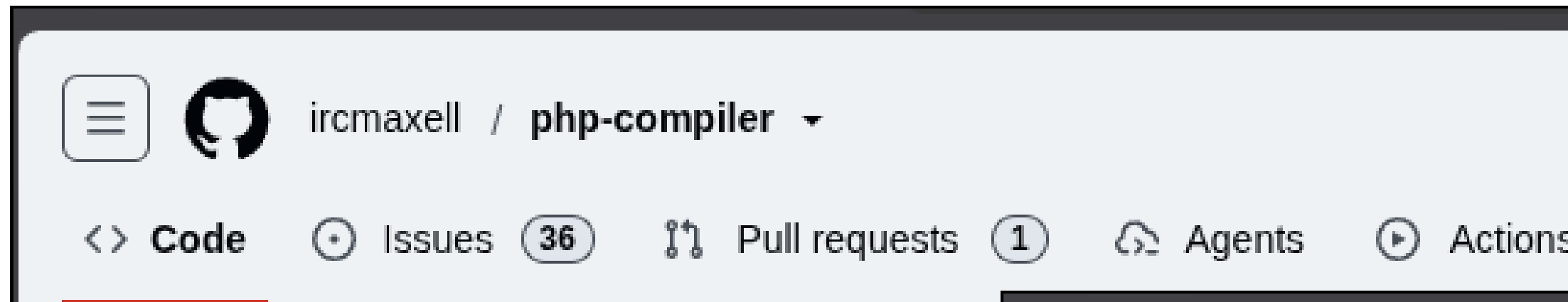
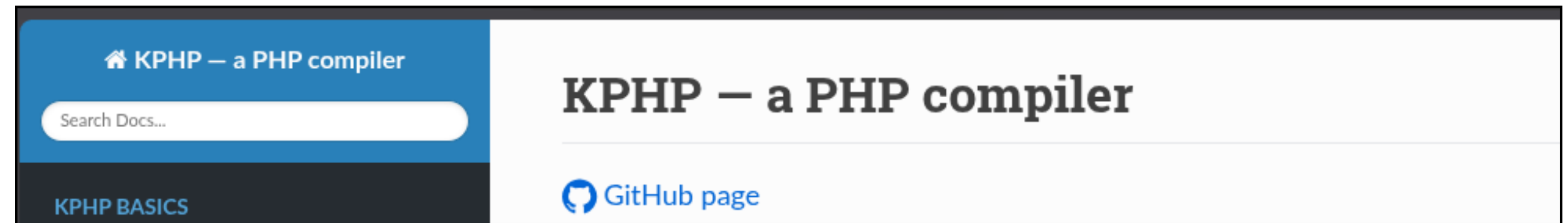
```
# Install the PHPX extension
cd vendor/swoole/phpx && cmake . && make -j$(nproc)

# Compile your first native binary
./tpc hello.php -O2 -o hello

# Run
./hello
```

PHP 8.6 and Other stuff

**Compiling PHP is a big (and controversial) topic,
worthy of a meet-up presentation!**



PHP 8.6 and Other stuff

**Fly.io is changing their business:
providing a cloud environment for
AI software development**

[/fly.io/blog/kurt-scott-money-sprites/](https://fly.io/blog/kurt-scott-money-sprites/)

Turn And Face The Strange



 [Annie Ruygt](#)

We're Fly.io, a public cloud platform that is both our favorite way to put an app on the Internet and our favorite way to safely let a frontier agent coding harness cook. This is a post about our company, the future, and Sprites, which are computers for agents that [you can check out right now](#).

This is a complicated post. So I need you to promise me something: if you read past this introduction, you'll read the whole rest of the way through. It's an honor thing.

IMO, this is embarrassing
for our PHP Community

IMO, people over there are
seeing their shares of
Automattic crater, and are
desperate to save whatever
value remains

BUSINESS & ENTERPRISE, WORDPRESS PROJECT

Matt Mullenweg Swears for the First Time in 42 Years and Posts Hiring Call to Move “My Stuff” Off Automattic Servers



Mullenweg broke a 42-year streak of not swearing, posted a hiring call on X, and told The Repository to “just wait and watch.”

(www.therepository.email/

[matt-mullenweg-swears-for-the-first-time-in-42-years-and-posts-hiring-call-to-move-my-stuff-off-automattic](#))

PHP 8.6 and Other stuff

What's known is, at least, some of the backdrop that's in the public domain:
BlackRock's latest SEC filing values its Automattic shares at \$14.33, an 83% decline from the \$85 it paid in 2021. The spoliation sanctions hearing is September 30. The company laid off 16% of its workforce in April 2025 and has been in federal litigation with WP Engine for nearly two years.

→ gonna lose

- 83% decline
- laid off 16%
- ongoing litigation - WP Enging's case has merit

PHP 8.6 and Other stuff



IMO, the biggest “tell” with WP:
no one wants to fork it

PHP 8.6 and Other stuff

“The part of WordPress that actually keeps enterprise IT up at night isn’t Automattic’s org chart. It’s WordPress.org, the plugin and theme directory every WordPress site pulls its security updates from, and the WordPress trademark,” Dickson told *Computerworld*.

IMO, WordPress has not integrated
Composer (and Packagist) on purpose.

In order to control the update mechanism.
Shocking how much \$\$ this represents.

IMO, WordPress has peaked.

**IMO, after a slow decline,
we will wake up one day to a mass exodus.**

HTTP Archive & "No CMS" GrowthData from the HTTP Archive (which analyzes millions of real mobile/desktop page loads) shows that the percentage of the web running "No Detectable CMS" rose from ~46.7% to over 48.3%. This confirms that new sites are increasingly built on modern JavaScript/static stacks rather than traditional database-driven CMSs like WordPress

PHP 8.6 and Other stuff

Well known
WordPress
forms plugin



0 items



Last updated: September 4, 2026

WordPress data & statistics 2026

How much of the web runs on WordPress, how that share is changing, and what actually powers WordPress sites, measured with first-party HTTP Archive data and refreshed every quarter.

Primary source: HTTP
Archive

Refreshed
quarterly



Casey Burrige

WordPress stats at a glance

PHP 8.6 and Other stuff

Shopify #2 !!

Means
WordPress
rejection
higher than
anyone is
admitting
out loud

WordPress stats at a glance

33.2%

Of all measurable web origins run WordPress
Source: HTTP Archive, April 2026

35.8%

WordPress's peak share, reached in July 2022 (now 33.21%)
Source: HTTP Archive

~64%

Of all sites with a detectable CMS run WordPress
Source: HTTP Archive, April 2026

32.7%

Of WordPress sites built with Elementor top page builder
Source: HTTP Archive, April 2026

Quick facts

- WordPress runs 33.21% of all measurable web origins as of April 2026, and about 64% of all sites that use a detectable CMS, more than every other CMS combined. ([Source](#))
- WordPress peaked at 35.76% in July 2022 and has declined 2.55 percentage points since, roughly a 7% relative drop over four years. ([Source](#))
- Elementor is the most-used WordPress page builder, detected on 32.67% of WordPress sites, followed by wpBakery at 8.52%. ([Source](#))
- Shopify is the second-biggest CMS at 4.74%, the only platform gaining share at scale (up 12.4% year over year), and it absorbed roughly 59% of the share WordPress lost this year. ([Source](#))
- Nearly half of the measurable web (48.3%) runs no detectable CMS at all, including static sites, headless builds, and hand-coded pages, up from 46.75% in mid-2024. ([Source](#))
- Contact Form 7 is the most-used WordPress form plugin, detected on 29.74% of WordPress sites in July 2026, though it has lost about a quarter of its share since 2022 (down from 39.06%). Gravity Forms (3.53%) is the only tracked form plugin whose share grew over the past year. ([Source](#))

Shopify + Astro
+ Cloudflare Pages

PHP 8.6 and Other stuff



Some announcements...

PHP 8.6 and Other stuff

Laravel LSP

The Laravel VS Code extension has quietly become one of the most useful tools in a Laravel developer's editor: autocomplete for routes, views, and config, inline documentation, and more. Until now, all of that lived inside one extension for a single editor.

Laravel LSP generalizes that same functionality into a Language Server Protocol implementation, so any editor that speaks LSP, including NeoVim, Zed, and Sublime Text, gets the same autocomplete and navigation features that VS Code users already have. Point your editor at the Laravel LSP, and it works the same way, regardless of which editor you use.

PHP 8.6 and Other stuff

Inertia DevTools

If you've used [Vue DevTools](#), Inertia DevTools will feel familiar. [Inertia DevTools](#) is a Chrome extension built specifically for Inertia applications that shows you every network request your app makes, how Inertia's headers are being set, and what props are hydrated on each page.

Instead of digging through the network tab to figure out why a prop isn't showing up the way you expect, Inertia DevTools gives you a dedicated view into how data flows between your Laravel backend and your Inertia frontend, end-to-end.

PHP 8.6 and Other stuff

Artisan dev

Most Laravel projects run their local dev processes through `composer dev`, which wraps `npx concurrently` to run your server, queue worker, and asset builder side-by-side. It works, but adding or removing a process meant editing your `composer.json` scripts by hand.

```
1 use Illuminate\Foundation\Console\DevCommands;
2
3 DevCommands::artisan('reverb:start', 'reverb')->orange();
4 DevCommands::register('stripe listen --forward-to ' . config('app.url'))-
```

You can also register additional commands from a service provider like Reverb or Stripe's CLI webhook listener (see code above)

The `artisan dev` command moves that configuration to the PHP side. You register your dev commands in your app service provider, assign each one a color (optional), and packages like [Laravel Reverb](#) or [Laravel Horizon](#) can register their own commands automatically the moment they're installed.

You control which processes run by chaining `only()` and `except()` methods when you register them, and you can inspect what's registered with the `--only` and `--except` flags on `artisan dev`. Under the hood, it uses `proc_open` to replace the PHP process directly, so there's no extra runtime overhead compared to the old `npx concurrently` approach.

PHP 8.6 and Other stuff

Head tag API

Managing everything between your HTML `<head>` tags, Open Graph images, canonical links, progressive web apps metadata, descriptions, has traditionally meant scattering `<meta>` tags across your Blade views by hand. The new head tag API gives you a fluent, PHP-side way to configure everything in one place.

Set defaults globally for your entire application, then override them at the route or view level wherever you need more specific behavior.

PHP 8.6 and Other stuff

Artisan doctor

Artisan doctor runs a set of health checks against your Laravel application: is your `APP_KEY` set, does your PHP version match what Composer expects, are your required extensions installed, and is your environment configuration complete? Where it can fix an issue automatically, it does. Where it can't, it tells you exactly what's wrong.

Packages can register their own diagnostic checks, so a package with specific configuration requirements can plug directly into `artisan doctor` and surface its own health checks alongside the framework's. It's also a natural last step for AI coding agents: after making changes, an agent can run `artisan doctor` as a final sanity check before considering a task done.

PHP 8.6 and Other stuff

Laravel image manipulation

Laravel's new image manipulation API gives you a fluent, chainable way to resize images, convert formats, set device pixel ratio, and output the result directly to the browser, all from PHP, without reaching for a separate image library.

PHP 8.6 and Other stuff

Laravel AI SDK: Human-in-the-Loop Tool approval

Until now, an [AI agent built with the Laravel AI SDK](#) ran on autopilot: once it started executing tools, there was no way to step in. The new [human-in-the-loop \(HITL\) API](#) lets you intercept specific agent actions and require a decision before the agent proceeds, so you can approve, deny, or modify what happens next instead of finding out after the fact.

Filesystem tools for AI agents

Agents built with the Laravel AI SDK can now use [built-in filesystem tools](#) to read, write, and manage files as part of their tool set, without you having to build that integration yourself.

Summarize any string

The Laravel AI SDK [adds a `summarize\(\)` method](#) on the `Str` helper, no configuration required. It uses the cheapest model available on your configured provider by default, and you can specify how many sentences you want back. Pair it with an Eloquent observer, and a model's summary field can update automatically whenever the underlying content changes.

Multimodal embeddings

The Laravel AI SDK can now [generate embeddings for images and audio](#), not just text, so you can build semantic search across media instead of relying on filenames or tags.



8.6

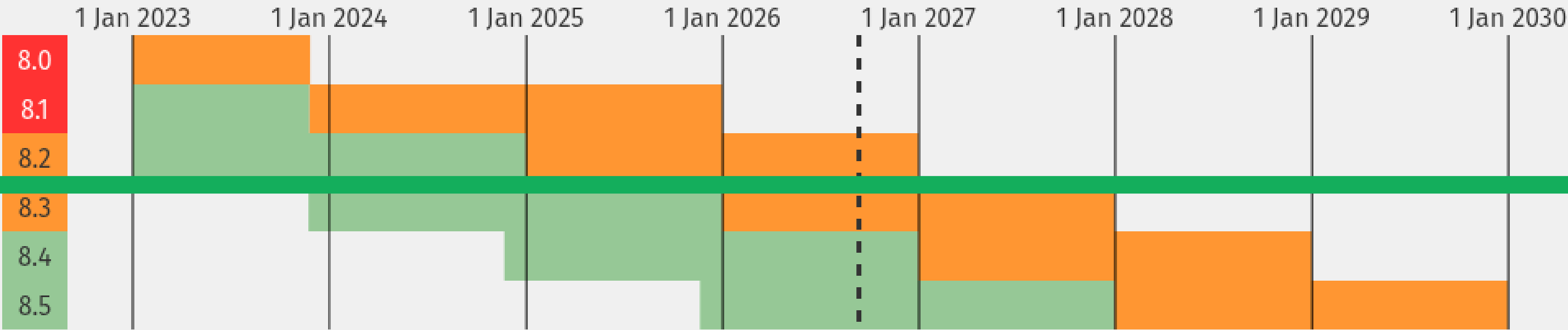
PHP 8.6 and Other stuff

Currently Supported Versions

(www.php.net/supported-versions.php)

Branch	Initial Release		Active Support Until		Security Support Until		
8.2	8 Dec 2022	3 years, 9 months ago	31 Dec 2024	1 year, 8 months ago	31 Dec 2026	in 3 months	Migration
8.3	23 Nov 2023	2 years, 9 months ago	31 Dec 2025	8 months ago	31 Dec 2027	in 1 year, 3 months	Migration
8.4	21 Nov 2024	1 year, 9 months ago	31 Dec 2026	in 3 months	31 Dec 2028	in 2 years, 3 months	Migration
8.5	20 Nov 2025	9 months ago	31 Dec 2027	in 1 year, 3 months	31 Dec 2029	in 3 years, 3 months	Migration

Or, visualised as a calendar:



Today: 12 Sep 2026

PHP 8.6 and Other stuff

php Login Register

start > todo > php86

PHP 8.6 Preparation Tasks

- Elect managers
- Triage GitHub issues for PHP 8.6
- Triage GitHub pull requests
- Push appropriate RFC at appropriate time

Timetable

Based upon a target GA in late November. Individual releases may be added or removed to accommodate development progress.

- Tag on Tuesday, release on Thursday
- 5 months pre-release phase

Date	Release
Jul 02 2026	Alpha 1 Announce Files
Jul 16 2026	Alpha 2 Announce Files

(www.php.net/todo/php86)

GA expected
late November

wiki.php.net/rfc#php_86

PHP 8.6 and Other stuff

PHP 8.6

- [Add pack\(\)/unpack\(\) endianness modifiers for floating-point numbers](#) (Created 2026-01-30)
- [Add pack\(\)/unpack\(\) support for signed integers with specific endianness](#) (Created 2025-09-15)
- [Stream Error Handling Improvements](#) (Created 2025-11-18)
- [Allow Object Property Writes on Objects Referenced by Constants](#) (Created 2026-02-24, discussion restarted 2026-06-28)
- [Limit maximum number of filter chains](#) (created 2026-05-05)
- [#\[\Override\] for class constants](#) (created 2026-03-13; voting started 2026-05-07, ended 2026-05-22)
- [Deprecate returning values from __construct\(\) and __destruct\(\)](#) (created 2026-05-08)
- [Add Locale::getDisplayKeyword and Locale::getDisplayKeywordValue functions](#) (created 2026-05-22)
- [Polling API](#) (Created 2025-10-30, voting started 2026-05-18)
- [Debugable Enums](#) (Created 2026-03-12; discussion started 2026-03-12; voting started 2026-04-02, ends 2026-04-16)
- [Secure Session Configuration Defaults](#) (Created 2026-04-04)

PHP 8.6 and Other stuff

- [Secure Session Configuration Defaults](#) (Created 2026-04-04)
- [Oniguruma maintenance end and end of mbregex](#) (Created 2025-08-21; voting started 2026-04-10, ends 2026-04-25)
- [enum SortDirection](#) (Created 2026-03-02)
- [DocComments For Function Parameters](#) (Created 2026-02-21)
- [Add Form Feed in Trim Functions](#) (Created 2025-12-29)
- [grapheme_strrev: strrev for grapheme cluster](#) (Created 2026-01-14)
- [Add "clamp\(\)" function](#) (Created 2025-08-21)
- [isReadable/Writeable Reflection methods](#) (Created 2024-11-28)
- [New function mysqli_quote_string](#) (Created 2025-12-18)
- [TLS Session Resumption Support for Streams](#) (Created 2025-12-22; discussion started 2025-12-22)
- [Display Function Arguments in Errors](#) (Created 2026-03-09; Accepted 2026-05-15)
- [Partial Function Application, v2](#) (Created 2025-06-03)
 - [Partial Function Application: Handling of Optional Parameters](#) (created 2026-04-20)
- [Duration class](#) (created 2026-06-16)
- [Closure optimizations](#) (Created 2026-01-30)
- [Readonly Property Defaults](#) (created 2026-07-04)
- [SNMP Improvements](#) (Created 2026-03-30; Accepted 2026-05-08)

PHP 8.6 and Other stuff

Add pack()/unpack() endianness modifiers for floating-point numbers

Screw around with the bits-and-bytes representation of numbers.

PHP is sending numbers to another system, device, network protocol.

Except that PHP is sending bytes, not a number.

And that thing receiving needs bytes represented in a specific way.

Someone needed signed integers with explicitly specified byte order, PHP couldn't do it cleanly, and the proposed solution was to add the missing syntax.

Add `pack()/unpack()` support for endianness modifiers on integers

Same thing as previous RFC.

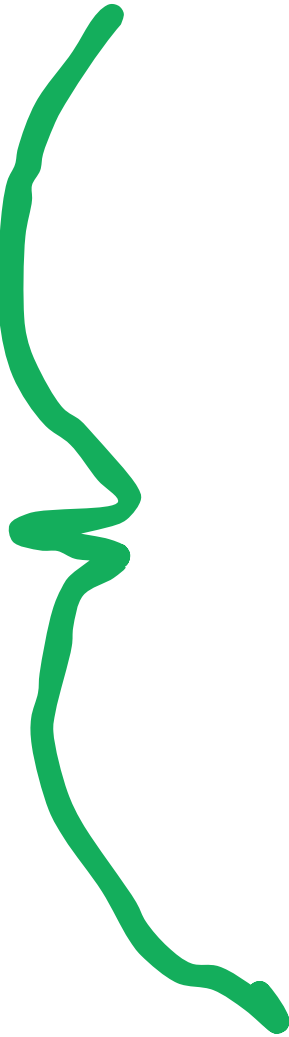
Previous RFC for floating point numbers.

This one is for integers.

PHP 8.6 and Other stuff

Stream Error Handling Improvements

types of
PHP
streams



- **Files** — reading from or writing to files.
- **Standard input/output** — stdin, stdout, and stderr.
- **Network connections** — TCP, UDP, and Unix sockets.
- **HTTP/HTTPS** — accessing web resources.
- **FTP** — accessing FTP resources.
- **PHP memory streams** — data held in memory.
- **Temporary streams** — data held in memory, or a temporary file.
- **Process pipes** — communicating with another running program.
- **Compressed streams** — reading/writing compressed files such as gzip.
- **User-defined streams** — PHP lets extensions and applications create their own stream types.

Different types of PHP streams encounter the same kinds of failure, but report it differently. Now, report it (more) consistently.

Allow Object Property Writes on Objects Referenced by Constants

types of
PHP
constants

- 
- **Scalar values:** int, float, string, bool
 - **Arrays**
 - **Objects** — since PHP 8.1, using new in constant expressions
 - **Other constant values** — for example, another constant

before RFC: constants that are objects, where the object has constant(s), these constant(s) could not be changed

after RFC: can be changed

PHP 8.6 and Other stuff

Limit maximum number of filter chains

“This RFC proposes to introduce a limit on the number of filters that can be chained within a `php://filter` URL. **The primary goal is to mitigate a class of security vulnerabilities** where Local File Inclusion (LFI) is elevated to Remote Code Execution (RCE) via complex, generated filter chains.

Currently, an attacker can use a long sequence of `convert.iconv.*` and `convert.base64` filters to craft arbitrary content (like a PHP shell) out of any file on the disk, or even from an empty resource. **By limiting the chain length to a reasonable number (e.g., 16), we can effectively disable these automated exploitation tools** while maintaining support for legitimate use cases.”

(Laravel Framework might be impacted)

PHP 8.6 and Other stuff

#[\Override] for class constants

**When a child class defines a constant
with the same name as a parent constant,
PHP could not tell
whether you intended to override it,
or simply created a new constant with that name.**

**#[\Override] lets you make that intention explicit,
so that PHP knows what you want to do.**

PHP 8.6 and Other stuff

Deprecate returning values from `__construct()` and `__destruct()`

PHP currently accepts “return value;” inside:

`__construct()`

`__destruct()`

PHP 8.6 will give a deprecation warning.

In a future major PHP version, it will become an error.

“return;” — with no value — allowed because it simply stops the constructor/destructor early.

Add `Locale::getDisplayKeyword()` and `Locale::getDisplayKeywordValue()`

PHP already has family of locale display helpers, such as:

- `Locale::getDisplayLanguage()`
- `Locale::getDisplayRegion()`
- `Locale::getDisplayVariant()`
- `Locale::getDisplayName()`
- `Locale::getKeywords()`

What is missing is a way to get individual keyword names/values.

This RFC gets these individual keyword names/values.

Debugable Enums

“__debugInfo()” controls what properties are displaying
in “var_dump()” and “print_r()”

before RFC: “__debugInfo()” not include enums.

after RFC: enums included.

PHP 8.6 and Other stuff

Secure Session Configuration Defaults

Three ini settings in PHP's session extension have security implications.

This RFC proposes changing those defaults so that new PHP installations are secure out of the box, without requiring application code changes for the common case.

The affected settings are:

- `session.use_strict_mode` — reject unrecognised session IDs supplied by the client
- `session.cookie_httponly` — exclude the session cookie from the JavaScript cookie API
- `session.cookie_samesite` — restrict cross-site delivery of the session cookie

Each change is a single-line edit to `ext/session/session.c`. No new ini entries, functions, or constants are introduced.

PHP 8.6 and Other stuff

Oniguruma maintenance end and end of mbregex

Oniguruma,
the library used by PHP's mbregex functions,
is no longer maintained.

This RFC deprecates mbregex in PHP 8.6,
and
removes it in PHP 9.0.

PHP 8.6 and Other stuff

enum SortDirection

This RFC proposes adding a single, always-available SortDirection enum to the global namespace:

```
enum SortDirection {  
    case Ascending;  
    case Descending;  
}
```

(TBH, I don't get this one. I am displaying it, and then moving on).

PHP 8.6 and Other stuff

DocComments For Function Parameters

**Add DocComment support for
function parameters,
and
a new function in the ReflectionParameter class.**

```
<?php
```

```
/** Search for entries matching query */
```

```
function search(
```

```
    /** Terms to search for in database */
```

```
    string $query,
```

```
    /** Maximum number of entries returned */
```

```
    int $num = 10
```

```
) {
```

```
    return ['foo', 'bar', 'qux'];
```

```
}
```

```
foreach (new ReflectionFunction("search")->getParameters() as $p) {
```

```
    echo $p->name . ": " . $p->getDocComment() . "\n";
```

```
}
```

```
?>
```

PHP 8.6 and Other stuff

Add Form Feed in Trim Functions

The trim(), ltrim(), and rtrim() functions strip whitespace (or other characters) from the beginning and end of a string.

When the second parameter (\$characters) is omitted, these functions use a default set of characters.

Currently, this default set includes:

- “ ” (ASCII 32 (0x20)), an ordinary space.
- “\t” (ASCII 9 (0x09)), a tab.
- “\n” (ASCII 10 (0x0A)), a new line (line feed).
- “\r” (ASCII 13 (0x0D)), a carriage return.
- “\0” (ASCII 0 (0x00)), the NUL-byte.
- “\v” (ASCII 11 (0x0B)), a vertical tab.

Notably missing from this list is “\f” (ASCII 12 (0x0C)), the Form Feed character.

This RFC proposes adding “\f” to the default list of characters stripped by trim(), ltrim(), and rtrim().

PHP 8.6 and Other stuff

grapheme_strrev: strrev for grapheme cluster

This proposal is strrev for grapheme cluster unit function.

Userland sometimes shows mb_strrev function, but mbstring(Unicode code point unit) can not support multi code point (for example, emoji etc...).

Almost same of Swift's “String.reversed()”.

PHP 8.6 and Other stuff

clamp

clamp checks if a comparable value is within a certain bound.

If the value is in range it returns the value.

If the value is not in range it returns the nearest bound.

clamp is already available in multiple languages
with a rather common implementation and prototype,
so this RFC aims to bring this feature natively in PHP also.

PHP 8.6 and Other stuff

isReadable/Writable reflection methods

The `ReflectionProperty::isPublic()` method indicates when a property has a “public” flag set on it, nothing more.

Prior to PHP 8.1, that implicitly also meant “can be written to from scope outside the object.”

However, PHP 8.1 introduced readonly properties, which broke that assumption with implicit private-set visibility.

The addition of explicit asymmetric visibility in PHP 8.4 further undermined that assumption. The result is that there is currently no straightforward way to determine at runtime if reading from or writing to a property would be allowed. This RFC attempts to provide such a utility.

PHP 8.6 and Other stuff

New function `mysqli_quote_string`

Adds new method `mysqli::quote_string()`.

Fixes last remaining SQL injection loophole.

Provides a manual escaping mechanism.

PHP 8.6 and Other stuff

TLS Session Resumption Support for Streams

This RFC proposes adding comprehensive TLS session resumption support to PHP's OpenSSL stream implementation.

Session resumption is a critical performance optimization that allows TLS clients and servers to skip the expensive full handshake by reusing cryptographic parameters from previous connections.

Currently, PHP's stream wrapper provides limited control over session management.

While OpenSSL's client session can be copied using `stream_socket_enable_crypto` `session_stream` parameter, developers cannot:

- Save and restore client sessions across PHP requests
- Implement custom server-side session storage (Redis, Memcached, databases)
- Control session cache behavior and use internal session cache (due to not re-using `SSL_CTX`)
-

This proposal exposes OpenSSL's session management APIs through the existing stream context options, providing fine-grained control while maintaining backward compatibility.

PHP 8.6 and Other stuff

Display Function Arguments in Errors

PHP errors (and warnings, notices, etc.) for built-in native functions show the function that raised them.

However, the display of parameters in these errors is inconsistent. In practice, only basic stream functions provide parameters, and those displayed parameters aren't derived from the argument list that the function was called with.

What this RFC proposes is including the parameters that were actually passed to that function, when that function raises an error.

Partial Function Application: Handling of Optional Parameters

Syntactic sugar.

Allow the placeholder symbol “?” to be omitted

or

to be handled implicitly
in certain short-closure or partial-application contexts.

PHP 8.6 and Other stuff

Duration class

PHP does not yet have a way to represent “stop-watch time” duration in a structured and intuitive way.

New Time\Duration class,

that represents durations
as measured by a “stop-watch”
or

an “egg-timer”

in a structured way with nanosecond precision.

Closure optimizations

This RFC proposes two new optimizations for closures (including arrow functions) that come with some theoretical BC breaks.

The purpose of this RFC is to evaluate whether these BC breaks are an acceptable trade-off for the gained performance.

PHP 8.6 and Other stuff

Readonly Property Defaults

This RFC seeks to allow default values for instance readonly properties.

Historically, default values on readonly properties were not disallowed for technical reasons, but because they were considered “not particularly useful”.

With interface properties, introduced in PHP 8.4, this changed.

Allowing default values on readonly class properties enables creating a strict contract for implementation property values that will not change at runtime -- as in, constant-like behaviour with contract.

SNMP improvements for encryption protocols, mib reset, and controlling output

This RFC proposes to improve the functionality of the SNMP extension in one or more of the following ways:

- Increase the number of SNMPv3 security protocols supported
- Allow the SNMP MIB to be reset
- Implement more MIB parsing and value output controls

PHP 8.6 and Other stuff

Polling API

New API.

Additional functionality
that the existing “stream_select()” function
does not provide.

Polling API

“ASync” in PHP
is a worthy topic for a
future meet-up topic

PHP 8.6 and Other stuff



Partial Function Application (v2)

**It seems to me
that this RFC
comes down to providing
“syntactic sugar”.
for “Partial Functions”.**

**“Partial Functions” in PHP
is a worthy topic for a
future meet-up topic**

PHP 8.6 and Other stuff

rejected
for 8.6

The screenshot shows a web browser displaying the PHP RFC: Bound-Erased Generic Types page. The browser's address bar shows the URL `wiki.php.net/rfc/bound_erased_generic_types`. The page header includes the PHP logo, 'Login', 'Register', and a search bar. The breadcrumb trail is 'start > rfc > bound_erased_generic_types'. The main heading is 'PHP RFC: Bound-Erased Generic Types'. Below it, a list of metadata is provided: Version: 0.22, Date: 2026-05-08, Author: Seifeddine Gmati, azjezz@carthage.software, Status: Declined, Implementation: <https://github.com/php/php-src/pull/21969>, and Discussion thread: <https://news-web.php.net/php.internals/130816>. The 'Introduction' section follows, explaining that this RFC adds generic type syntax to PHP, allowing classes, interfaces, traits, functions, methods, closures, and arrow functions to declare type parameters with bounds, defaults, and variance markers. A code block at the bottom shows a partial PHP class definition: `<?php final readonly class Pair<+L, +R> { public function __construct( public L $left,`. A 'Copy' button is visible next to the code. On the right side, a 'Table of Contents' sidebar lists various topics, including 'PHP RFC: Bound-Erased Generic Types', 'Introduction', 'What are generics', 'Why people use generics', and several examples like 'Laravel: a doubly-generic collection' and 'Symfony: a covariant user provider'.

php Login Register Search

start > rfc > bound_erased_generic_types

PHP RFC: Bound-Erased Generic Types

- Version: 0.22
- Date: 2026-05-08
- Author: Seifeddine Gmati, azjezz@carthage.software
- Status: Declined
- Implementation: <https://github.com/php/php-src/pull/21969>
- Discussion thread: <https://news-web.php.net/php.internals/130816>

Introduction

This RFC adds generic type syntax to PHP. Classes, interfaces, traits, functions, methods, closures, and arrow functions can declare type parameters; those parameters carry bounds, defaults, and variance markers; type arguments may be supplied at use sites and at call sites via turbofish.

```
<?php

final readonly class Pair<+L, +R> {
    public function __construct(
        public L $left,
```

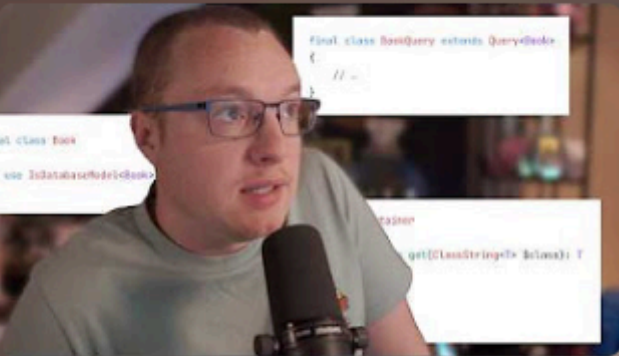
Copy

Table of Contents

- PHP RFC: Bound-Erased Generic Types
- Introduction
- What are generics
- Why people use generics
 - Laravel: a doubly-generic collection
 - Symfony: a covariant user provider
- PHP Standard Library: a multi-parameter graph interface
- Doctrine ORM: a generic method on a non-generic interface
- Tempest: per-method generics on a cache interface
- API Platform: implementing a generic interface
- PHPUnit: a constraint that returns its input
- Seven shapes, one syntax outside the language
- The cost
- Scale
- Userland preprocessors
- The projects
- PHP has no build step

**“Generics” in PHP
is a DEFINITE topic for a
future meet-up**

PHP 8.6 and Other stuff

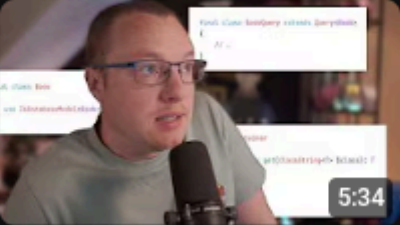


Generics in PHP

#1 by PHP Annotated

Playlist • 9 videos • 14,998 views

▶ Play all



Generics are not coming to PHP

PHP Annotated

3.2k views • 2 months ago

5:34



The final boss. Generics in PHP

PHP Annotated

5.5k views • 3 months ago

23:50



Generics in PHP???

PHP Annotated

8k views • 2 years ago

54:31



Not possible!

PHP generics aren't coming...

PHP Annotated

10k views • 2 years ago

22:40



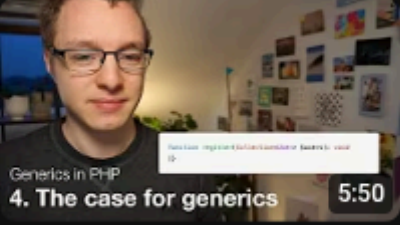
TypeScript, and... PHP?

PHP should be more like TypeScript

PHP Annotated

10k views • 4 years ago

8:19



The Case for PHP Generics — G

PHP Annotated

5.6k views • 4 years ago

5:50



Why we can't have generics in PHP

PHP 8.6 and Other stuff

The PHP Foundation

The PHP Foundation

Roman Pronskiy Leaves The PHP Foundation Board and Brent Roose Joins

Published on Sep 01, 2026 Elizabeth Barron Team News



The PHP Foundation is excited to announce the addition of [Brent Roose](#) from JetBrains to the Foundation's Board of Directors. Brent fills the JetBrains Platinum Sponsor seat and was unanimously voted to join by the existing members.

Brent is a developer advocate for PHP at JetBrains, but he is also a writer, content creator, event organizer, community leader, and open source contributor. He is a well-recognized voice on [PHP Annotated](#), maintains an active blog at [stitcher.io](#), and is the creator of the [Tempest](#) framework for PHP, one of his bigger open source projects.

PHP has been my passion for over 15 years, and I'm happy to contribute to its success in any way I can. I'm looking forward to joining the Foundation board and see what the future holds for this amazing language and community. --Brent Roose

As Roman Pronskiy's term comes to an end, Brent will be replacing Roman as the JetBrains representative on the Board. As one of the founding members and first Executive Director of The PHP Foundation, Roman's impact is undeniable. The PHP Foundation would not be where it is today without his influence, his effort, and his guidance. We are incredibly grateful for his dedication and commitment to the Foundation's mission and the sustainability of the PHP language. Thank you for everything, Roman! ❤️🐘

PHP 8.6 and Other stuff

